

Des obstacles pour un Lama

Un lama aimerait traverser l'altiplano. Il possède une carte du plateau représentée par une grille de $N \times M$ cellules. Les lignes de cette carte sont numérotées de 0 à $N - 1$ de haut en bas, et les colonnes sont numérotées de 0 à $M - 1$ de gauche à droite. La cellule de la carte dans la ligne i et colonne j ($0 \leq i < N, 0 \leq j < M$) est notée (i, j) .

Le lama a étudié le climat du plateau et a découvert que toutes les cellules d'une même ligne ont la même **température** et que toutes les cellules d'une même colonne ont la même **humidité**. Le lama vous donne deux tableaux d'entiers T et H respectivement de tailles N et M . Ici, $T[i]$ ($0 \leq i < N$) est la température de toutes les cellules de la ligne i , et $H[j]$ ($0 \leq j < M$) est l'humidité de toutes les cellules de la colonne j .

Le lama a aussi étudié la flore présente sur le plateau et a remarqué que la cellule (i, j) est **dégagée** si et seulement si sa température est strictement supérieure à son humidité, formellement, $T[i] > H[j]$.

Le lama ne peut traverser le plateau qu'en empruntant un **chemin valide**. Un chemin valide est une suite de cellules distinctes qui satisfont les conditions suivantes :

- Chaque paire de cellules consécutives partage un côté commun.
- Toutes les cellules du chemin sont dégagées.

Votre tâche est de répondre à Q questions. Pour chaque question, on vous fournit quatre entiers L, R, S et D . Vous devez déterminer s'il existe un chemin valide tel que :

- Le chemin commence sur la cellule $(0, S)$ et termine sur la cellule $(0, D)$.
- Toutes les cellules du chemin se situent entre les colonnes L et R incluses.

Il est garanti que $(0, S)$ et $(0, D)$ sont toutes les deux dégagées.

Détails d'implémentation

La première fonction à implémenter est :

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : un tableau de taille N spécifiant la température de chaque ligne.
- H : un tableau de taille M spécifiant l'humidité dans chaque colonne.

- Cette fonction est appelée exactement une fois pour chaque test, avant tout appel à `can_reach`.

La deuxième fonction à implémenter est la suivante :

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : entiers décrivant une question.
- Cette fonction est appelée Q fois pour chaque test.

Cette fonction doit renvoyer `true` si et seulement s'il existe un chemin valide entre la cellule $(0, S)$ et la cellule $(0, D)$, tel que toutes les cellules du chemin se trouvent dans les colonnes L à R , incluses.

Contraintes

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ pour tout i tel que $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ pour tout j tel que $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Les deux cellules $(0, S)$ et $(0, D)$ sont dégagées.

Sous-tâches

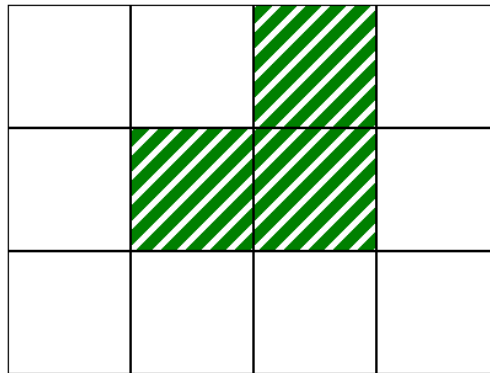
Sous-tâche	Score	Contraintes supplémentaires
1	10	$L = 0, R = M - 1$ pour chaque question. $N = 1$.
2	14	$L = 0, R = M - 1$ pour chaque question. $T[i - 1] \leq T[i]$ pour tout i tel que $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ pour chaque question. $N = 3$ et $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ pour chaque question. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ pour chaque question.
6	17	Aucune contrainte supplémentaire.

Exemple

Considérons l'appel suivant :

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

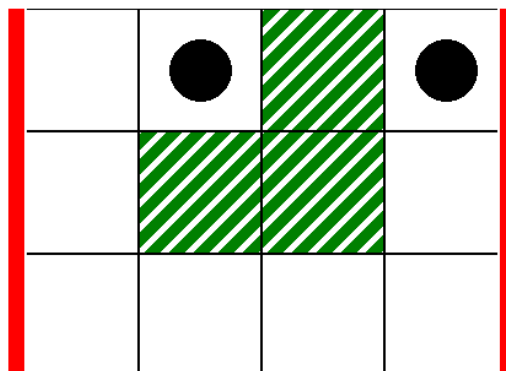
Cela correspond à la carte de l'image suivante, où les cellules blanches sont dégagées :



Pour la première question, considérez l'appel suivant :

```
can_reach(0, 3, 1, 3)
```

Cela correspond au scénario de l'image suivante, où les lignes verticales épaisses indiquent l'intervalle de $L = 0$ à $R = 3$, et les ronds noirs indiquent les cellules de départ et d'arrivée :



Dans ce cas, le lama peut se rendre de la cellule $(0,1)$ à la cellule $(0,3)$ par le chemin valide suivant :

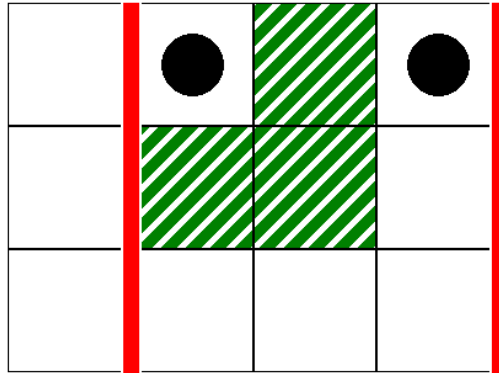
$(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)$

Par conséquent, cet appel doit renvoyer `true`.

Pour la deuxième question, considérez l'appel suivant :

```
can_reach(1, 3, 1, 3)
```

Cela correspond au scénario de l'image suivante :



Dans ce cas, il n'existe pas de chemin valable de la cellule $(0, 1)$ à la cellule $(0, 3)$ tel que toutes les cellules du chemin se trouvent dans les colonnes 1 à 3, incluses. Par conséquent, cet appel doit renvoyer `false`.

Évaluateur d'exemple (grader)

Format d'entrée :

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Ici, $L[k]$, $R[k]$, $S[k]$ et $D[k]$ ($0 \leq k < Q$) spécifient les paramètres pour chaque appel à `can_reach`.

Format de sortie :

```
A[0]
A[1]
...
A[Q-1]
```

Ici, $A[k]$ ($0 \leq k < Q$) est 1 si l'appel `can_reach(L[k], R[k], S[k], D[k])` a renvoyé `true`, et 0 sinon.